

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a5b909180a84cd78d.jsonl`  
- SHA-256: `f9d9d228966207c2f42f3ff2fed013ab7fc5bb95eed87e48e19612fc4e50a6fe`  
- Source modified: `2026-06-10T06:08:28+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_2a64bc88-2a8`  
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

Transcript

1. user (2026-06-10T06:06:42.247Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

ROLE: adversarial verifier. An auditor ("testing") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.

FINDING: {"title":"wasb_hybrid segmenter is an untested near-clone of tracknet_hybrid ? the known copy-paste-drift regression class ships green","severity":"high","area":"segmenters","file":"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/wasb_hybrid.py (16% coverage, 127/151 stmts missed)","evidence":"No test file imports backend.pipeline.segmenters.wasb_hybrid (grep over tests/ matches only string literals in test_reporting_harvest.py config dicts). Its sibling tracknet_hybrid.py has 8 behavioral tests (tests/test_tracknet_hybrid.py: happy path, no-API-key, healthcheck abort, no-trajectory abort, store_candidates flag, N/A timestamp dropping) and sits at 90% coverage. A token-normalized diff of the two files confirms they share an identical three-phase structure with already-diverging bodies (different error-log text, tn_cfg vs wasb_cfg config-key handling, docstrings); wasb_hybrid.py's own docstring says 'this is the live shuttle-motion substrate'.","why_it_matters":"Per project docs/memory, WASB is the production-favored detector (pretrained badminton weights, transfers to table tennis). Any regression in its candidate persistence, linking, store_yolo_candidates handling, or bad-timestamp dropping ? including drift introduced by editing only the tracknet twin ? passes the suite. The existing tracknet_hybrid tests are nearly copy-portable, so closing this is low-cost."}

Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:06:50.992Z)

```
backend\pipeline\segmenters\__init__.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
```

3. user (2026-06-10T06:06:52.083Z)

```
tests\test_calibrate_wasb.py:1: """Tests for the WASB calibration driver (pure parts; no
WSL/GPU)."""
tests\test_calibrate_wasb.py:5: from backend.eval.calibrate_wasb import (
tests\test_export_wasb_segments.py:4: from backend.eval import export_wasb_segments as ex
tests\test_rally_slicer.py:79:         {"id": 1, "start_time": 10.0, "end_time": 15.0,
"duration": 5.0, "metadata": {"shot_count": 4, "detector": "wasb-hybrid-gemini"}},
tests\test_rally_slicer.py:80:         {"id": 2, "start_time": 30.0, "end_time": 33.5,
"duration": 3.5, "metadata": {"shot_count": 3, "detector": "wasb-hybrid-gemini"}},
tests\test_reporting_harvest.py:46:         config={"ai_provider": "gemini", "ai_model": "gemini-
flash", "indexing": {"default_segmenter": "wasb_hybrid"}},
tests\test_reporting_harvest.py:49:         play_segments=[{"duration": 5.0, "metadata":
{"detector": "wasb-hybrid-gemini"}},
tests\test_reporting_harvest.py:51:         segmenter_requested="wasb_hybrid",
tests\test_reporting_harvest.py:52:         segmenter_actual="wasb_hybrid",
tests\test_reporting_harvest.py:107:    cfg = {"ai_provider": "gemini", "indexing":
{"default_segmenter": "wasb_hybrid"}}
tests\test_reporting_harvest.py:147:    db = _FakeDB(play=[{"duration": 4.0, "metadata":
{"detector": "wasb-hybrid"}}, candidates=[{"id": 1}])
tests\test_reporting_harvest.py:150:        config={"ai_provider": "gemini", "indexing":
{"default_segmenter": "wasb_hybrid"}},
tests\test_reporting_harvest.py:152:        segmenter_requested="wasb_hybrid",
segmenter_actual="wasb_hybrid",
tests\test_reporting_harvest.py:169:    db = _FakeDB(play=[{"duration": 4.0, "metadata":
{"detector": "wasb-hybrid"}}, candidates=[{"id": 1}])
tests\test_reporting_harvest.py:174:        segmenter_requested="wasb_hybrid",
segmenter_actual="wasb_hybrid",
tests\test_tracknet_runner.py:17: from backend.pipeline.detectors.wasb_runner import WasbRunner,
WasbConfig
tests\test_tracknet_runner.py:218: def test_wasb_runner_implements_detector_runner_abc():
tests\test_tracknet_runner.py:219:     """WasbRunner must also satisfy the DetectorRunner
contract."""
tests\test_tracknet_runner.py:220:     w = WasbRunner(WasbConfig())
tests\test_wasb_cache.py:1: """Resumable-cache layer of wasb_infer (pure-Python parts; no
WSL/GPU).
tests\test_wasb_cache.py:13: from backend.pipeline.detectors import wasb_infer as wi
tests\test_wasb_cache.py:159:     assert osp.basename(cd) == "sample_long_traj_wasbcache"
```

4. user (2026-06-10T06:07:01.184Z)

```
README.md:219: [Omitted long matching line]
docs\BACKLOG.md:79: - ? **Item 7 (unify hybrid segmenters) should land before the report's real
AI-provider timing.** The report records AI timing as `not_captured`; once
`wasb_hybrid`/`tracknet_hybrid` are unified there is ONE place to thread a recorder callback
through `provider.analyze_video` ? the report. **Revisit:** with item 7 (Claude to add the
timing hook there).
docs\COMMERCIALIZATION.md:171: shared logic from `tracknet_hybrid`/`wasb_hybrid` before heavy
AI timing/metrics land.
docs\CODE_MAP.md:38: Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid /
wasb_hybrid ? same shape):
docs\CODE_MAP.md:48: ### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
docs\CODE_MAP.md:50: WasbHybridSegmenter.process_video
@backend/pipeline/segmenters/wasb_hybrid.py
docs\CODE_MAP.md:109: - `@backend/pipeline/segmenters/__init__.py` ? imports all 5
```

```
(`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid`) = the de-facto registration manifest;
re-exports them + `segmenter_registry` via `__all__`.
docs\CODE_MAP.md:110:[Omitted long matching line]
docs\CODE_MAP.md:111:-
`@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`'tracknet_hybrid'`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig`
from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`;
`detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably
yolo_hybrid).
docs\CODE_MAP.md:177:[Omitted long matching line]
docs\CODE_MAP.md:251:- **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-
paste; fixes to P3/P4 must land in both. ? subtle drift: wasb's shot_count `int(None)`-via-
TypeError fallback vs tracknet's `is None` check.
docs\CODE_MAP.md:328:| (no dedicated test) | individual `pipeline/validators/*` impls,
`pipeline/stitcher.compiler`, `main.py`, `run_process_and_stitch.py`,
`pipeline/detectors/wasb_runner`, `pipeline/segmenters/{wasb_hybrid,motion}`,
`eval/audio/e1_probe` (diagnostic) ? ? exercised only indirectly |
docs\archives\COMMERCIAL_READINESS_REVIEW.md:65: - Extract shared logic from `tracknet_hybrid`
/ `wasb_hybrid` (and yolo if relevant) before heavy AI timing/metrics land in reports.
backend\reporting\harvest.py:22:_AI_SEGMENTERS = {"yolo_hybrid", "tracknet_hybrid",
"wasb_hybrid", "gemini"}
backend\eval\gemini_refine.py:14:This mirrors wasb_hybrid's Phase-3 handoff but runs standalone
on a cached
tests\test_reporting_harvest.py:46: config={"ai_provider": "gemini", "ai_model": "gemini-
flash", "indexing": {"default_segmenter": "wasb_hybrid"}},
tests\test_reporting_harvest.py:51: segmenter_requested="wasb_hybrid",
tests\test_reporting_harvest.py:52: segmenter_actual="wasb_hybrid",
tests\test_reporting_harvest.py:107: cfg = {"ai_provider": "gemini", "indexing":
{"default_segmenter": "wasb_hybrid"}}
tests\test_reporting_harvest.py:150: config={"ai_provider": "gemini", "indexing":
{"default_segmenter": "wasb_hybrid"}},
tests\test_reporting_harvest.py:152: segmenter_requested="wasb_hybrid",
segmenter_actual="wasb_hybrid",
tests\test_reporting_harvest.py:174: segmenter_requested="wasb_hybrid",
segmenter_actual="wasb_hybrid",
backend\pipeline\detectors\base.py:5:the high-accuracy shuttle trajectory substrate used by
tracknet_hybrid / wasb_hybrid.
backend\pipeline\detectors\base.py:64:-
backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
backend\pipeline\segmenters\__init__.py:6:from backend.pipeline.segmenters.wasb_hybrid import
WasbHybridSegmenter
backend\pipeline\segmenters\__init__.py:15: "WasbHybridSegmenter",
backend\pipeline\segmenters\wasb_hybrid.py:38:class WasbHybridSegmenter(BasePlaySegmenter):
backend\pipeline\segmenters\wasb_hybrid.py:41: return "wasb_hybrid"
backend\pipeline\segmenters\wasb_hybrid.py:99: "detector": "wasb_hybrid",
backend\pipeline\segmenters\wasb_hybrid.py:146: video_id,
detector="wasb_hybrid", start_time=cw["start"], end_time=cw["end"],
backend\pipeline\segmenters\wasb_hybrid.py:225:segmenter_registry.register(WasbHybridSegmenter)
```

5. user (2026-06-10T06:07:01.504Z)

```
1      """WASB-based hybrid play segmenter.
2
3      Identical three-phase shape to TrackNetHybridSegmenter, but Phase 2 uses **WASB**
4      (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via WSL2
5      (backend/detectors/wasb_runner.py) instead of TrackNetV4. WASB ships pretrained
6      badminton weights and runs today, so this is the live shuttle-motion substrate.
7      Phase 3 (AI semantic boundaries) and Phase 4 (DB) are shared with the other hybrids.
8
9      For the per-video *calibration* goal we mainly use Phase 2 (trajectory -> candidate
10     windows); the windowing thresholds (velocity_thresh / merge_gap / min_window_duration)
11     are what get tuned against golden rally labels.
12     """
13     import os
14     import time
```

```

15     import logging
16     import concurrent.futures
17     from typing import List, Dict, Any
18
19     import cv2
20
21     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
22     from backend.utils.video import get_video_duration, extract_video_chunk
23     from backend.sports import sport_registry
24     from backend.providers import provider_registry
25     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
26     from backend.pipeline.detectors.base import DetectorRunner # ABC for the de-risked
detector seam
27
28     logger = logging.getLogger(__name__)
29
30
31     def _fmt_ts(seconds: float) -> str:
32         seconds = max(0, int(seconds))
33         h, rem = divmod(seconds, 3600)
34         m, s = divmod(rem, 60)
35         return f"{h:02d}:{m:02d}:{s:02d}"
36
37
38     class WasbHybridSegmenter(BasePlaySegmenter):
39         @property
40         def name(self) -> str:
41             return "wasb_hybrid"
42
43         @property
44         def description(self) -> str:
45             return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
46                     "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
47
48         @staticmethod
49         def _probe_fps_width(video_path: str) -> tuple:
50             cap = cv2.VideoCapture(video_path)
51             fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
52             width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
53             cap.release()
54             return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
55
56         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None,
57                         max_frames: int = None) -> List[Dict[str, Any]]:
58             sport_name = self.config.get("sport", "badminton")
59             try:
60                 sport_profile = sport_registry.get(sport_name)
61             except KeyError as e:
62                 logger.error(str(e))
63                 return []
64
65             idx_cfg = self.config.get("indexing", {})
66             provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
67             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
68
69             api_key_env_var = f"{provider_name.upper()}_API_KEY"
70             api_key = os.environ.get(api_key_env_var)
71             if not api_key:
72                 logger.error(f"{api_key_env_var} not set; cannot run the {provider_name}
handoff.")

```

```

73         return []
74     try:
75         provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
76     except KeyError as e:
77         logger.error(str(e))
78         return []
79
80     video_duration = get_video_duration(video_path)
81     if video_duration <= 0:
82         logger.error("Invalid video duration.")
83         return []
84     fps, frame_width = self._probe_fps_width(video_path)
85
86     wasb_cfg = WasbConfig.from_indexing_cfg(idx_cfg)
87     runner: DetectorRunner = WasbRunner(wasb_cfg)
88
89     w = idx_cfg.get("wasb", {})
90     velocity_thresh = w.get("velocity_threshold", 0.02)
91     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
92     min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
93     handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
94     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
95     log_candidates = idx_cfg.get("log_yolo_candidates", True)
96     store_candidates = idx_cfg.get("store_yolo_candidates", True)
97
98     detection_params = {
99         "detector": "wasb_hybrid",
100         "velocity_thresh": velocity_thresh,
101         "merge_gap": merge_gap,
102         "min_action_window_duration": min_window_duration,
103         "weights_path": wasb_cfg.weights_path,
104     }
105
106     ok, msg = runner.healthcheck()
107     if not ok:
108         logger.error("WASB WSL environment not ready: %s", msg)
109         return []
110     logger.info("WASB WSL env OK: %s", msg)
111
112     # --- Phase 2: shuttle trajectory -> candidate rally windows ---
113     t_start = time.time()
114     out_dir = os.path.join("output", "wasb")
115     csv_path = runner.run_predict(video_path, out_dir)
116     if not csv_path:
117         logger.error("WASB produced no trajectory; aborting.")
118         return []
119
120     points = runner.parse_trajectory_csv(csv_path)
121     logger.info("Parsed %d trajectory points (%d visible).",
122                 len(points), sum(1 for p in points if p.visible))
123
124     all_candidate_windows = runner.trajectory_to_action_windows(
125         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
126         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
127         min_window_duration=min_window_duration, chunk_index=0,
128     )
129     logger.info("Phase 2 (WASB) completed in %.1fs. Candidate windows: %d",
130                 time.time() - t_start, len(all_candidate_windows))
131
132     if log_candidates:
133         for cw in all_candidate_windows:
134             logger.info(f"WASB rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
135                         f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "

```

```

136                 f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")
137
138         candidate_ids = [None] * len(all_candidate_windows)
139         if store_candidates:
140             for i, cw in enumerate(all_candidate_windows):
141                 stats = {
142                     "peak_velocity": cw["peak_velocity"], "mean_velocity":
cw["mean_velocity"],
143                     "active_frame_count": cw["active_frame_count"], "track_id_count":
cw["track_id_count"],
144                 }
145                 candidate_ids[i] = self.db.add_candidate_segment(
146                     video_id, detector="wasb_hybrid", start_time=cw["start"],
end_time=cw["end"],
147                     chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
148                     stats=stats, detection_params=detection_params,
149                 )
150
151         if not all_candidate_windows:
152             return []
153
154         # --- Phase 3: AI provider handoff (shared pattern) ---
155         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
156         os.makedirs(handoff_dir, exist_ok=True)
157         logger.info("Phase 3: %s validation on %d candidate windows...",
158                     provider_name, len(all_candidate_windows))
159
160         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
161             w_start = max(0, window["start"] - handoff_padding)
162             w_end = min(video_duration, window["end"] + handoff_padding)
163             w_dur = w_end - w_start
164             w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
165             if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
166                 return []
167             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
168             segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
169                                                  label=f"Handoff {wi+1}")
170             valid = []
171             for seg in segments:
172                 try:
173                     seg["start_time"] = float(seg["start_time"]) + w_start
174                     seg["end_time"] = float(seg["end_time"]) + w_start
175                     seg["_candidate_id"] = candidate_ids[wi]
176                     valid.append(seg)
177                 except (ValueError, TypeError, KeyError) as e:
178                     logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
179             try:
180                 os.remove(w_path)
181             except OSError:
182                 pass
183             return valid
184
185         ai_segments: List[dict] = []
186         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
187             futures = [executor.submit(process_candidate_window, wi, cw)
188                         for wi, cw in enumerate(all_candidate_windows)]
189             for future in futures:
190                 ai_segments.extend(future.result())
191         try:
192             os.rmdir(handoff_dir)

```

```

193         except OSError:
194             pass
195         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
196 len(ai_segments))
197
198         # --- Phase 4: DB population (shared pattern) ---
199         segments_data = []
200         for segment in ai_segments:
201             try:
202                 start = float(segment["start_time"])
203                 end = float(segment["end_time"])
204                 dur = end - start
205                 shot_count = segment.get("shuttle_exchange_count")
206                 try:
207                     shot_count = int(shot_count)
208                 except (ValueError, TypeError):
209                     shot_count = max(3, int(dur / 0.8))
210                 metadata = {"shot_count": shot_count,
211                             "shot_count_confidence":
212 segment.get("shot_count_confidence", "Unknown"),
213                             "detector": f"wasb-hybrid-{model_name}"}
214                 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
215 metadata)
216                 if segment_id:
217                     candidate_id = segment.get("_candidate_id")
218                     if candidate_id is not None:
219                         self.db.link_candidate_to_segment(candidate_id, segment_id)
220                     segments_data.append({"id": segment_id, "start_time": start,
221 "end_time": end,
222 "duration": dur, "metadata": metadata})
223             except Exception as e:
224                 logger.warning("Skipping invalid segment struct: %s", e)
225
226         return segments_data
227
228     segmenter_registry.register(WasbHybridSegmenter)
229

```

6. user (2026-06-10T06:07:21.291Z)

```

47: def test_pipeline_happy_path(mock_runner_cls, mock_probe, mock_dur, mock_extract,
76: def test_pipeline_no_api_key_returns_empty(mock_runner_cls, mock_probe, mock_dur,
mock_extract,
87: def test_pipeline_healthcheck_failure_aborts(mock_runner_cls, mock_probe, mock_dur,
mock_extract,
103: def test_pipeline_no_trajectory_aborts(mock_runner_cls, mock_probe, mock_dur, mock_extract,
115: def test_pipeline_store_candidates_disabled(mock_runner_cls, mock_probe, mock_dur,
mock_extract,
137: def test_pipeline_drops_unparseable_segments(mock_runner_cls, mock_probe, mock_dur,
mock_extract,
158: def test_probe_fps_width_fallbacks():
168: def test_segmenter_identity():

```

7. user (2026-06-10T06:07:22.011Z)

25:- ? **``yolo\_hybrid`` naming** ? the registry key, **``yolo\_``** config keys, and **``yolo-hybrid-``** DB detector tags keep the "yolo" name for back-compat despite no YOLO remaining. Optional rename. **``Revisit``** if/when a clean break is worth the migration.

79:- ? **Item 7 (unify hybrid segmenters)** should land before the report's real AI-provider timing. **``The report records AI timing as ``not_captured``; once ``wasb_hybrid``/``tracknet_hybrid`` are unified there is ONE place to thread a recorder callback through ``provider.analyze_video`` ? the report.``** **``Revisit``** with item 7 (Claude to add the timing hook there).

8. user (2026-06-10T06:07:41.667Z)

```
31: - `[project.optional-dependencies] dev` ? `pytest`, coverage tooling.
39: shadows the build system; update README/`run_all_tests.py`/CI references.
50:- `pip install -e .[dev]` in a clean venv ? `run_all_tests.py` green.
59:`calibrate_wasb.py` do honest leave-one-video-out (LOVO), but nothing *enforces*
85:- Unit test: overlapping train/eval ids ? `assert_disjoint` raises.
```

9. user (2026-06-10T06:07:42.186Z)

```
315 | `test_gemini_refine.py` | `eval.gemini_refine` merge_overlaps / _offset_segments (pure
helpers, no API) |
316 | `test_hit_detector.py` | `pipeline/audio/hit_detector` (detect_hits, onset_envelope,
_moving_stats) + `eval/audio/e1_probe.cluster_hits` |
317 | `test_rally_slicer.py` | `tools.rally_slicer.compute_clip_windows` (pure) + label
parse |
318 | `test_video.py` | `utils.video` duration/chunk (subprocess mocked) |
319 | `test_geometry.py` | `utils.geometry` center/distance/velocity/normalised-velocity/IoU
|
320 | `test_database.py` | `infrastructure.database.Database`
schema/CRUD/migrations/`user_version`/`EXPECTED_CANDIDATE_COLUMNS` guard |
321 | `test_config.py` | `config` (load_config defaults, save_default_config, AppConfig
defaults pinned, `extra` tolerance, `.get()` dict-compat) |
322 | `test_reporting_harvest.py` | `reporting` (`emit_indexer_report`,
`harvest.record_run`); ? `importorskip("obsreport")`; all 7 footgun rules + plain-dict-OR-
AppConfig + `report.enabled=False` |
323 | `test_reporting_spec_faq.py` | `reporting/spec.yaml` <-> `README.md` cross-ref
integrity (every `faq_ref` resolves; actionable rules cite a `faq_ref`); ?
`importorskip("obsreport")` |
324 | `test_validators_base.py` | `pipeline/validators/base.ValidationRegistry`
(register/run_all/`run_all_with_context` -> `(results,context)`; exception captured not raised)
|
325 | `test_sport_profiles.py` | `sports` registry + BadmintonSportProfile (prompt contains
"decimal seconds"/chunk value; relevance config keys) |
326 | `test_providers.py` / `test_provider_gemini.py` | `providers` registry +
GeminiProvider lifecycle (`MAX_UPLOAD_MB` guard, non-list JSON -> [], cleanup delete) |
327 | `test_annotation_providers.py` / `test_automated_provider.py` | `annotations`
registry, FileProvider, AutomatedProvider (oracle/lineage); keystone one-scorer contract ->
`eval.metrics.score` |
328 | (no dedicated test) | individual `pipeline/validators/*` impls,
`pipeline/stitcher.compiler`, `main.py`, `run_process_and_stitch.py`,
`pipeline/detectors/wasb_runner`, `pipeline/segmenters/{wasb_hybrid,motion}`,
`eval/audio/e1_probe` (diagnostic) ?? exercised only indirectly |
329 | ? stale `.pyc` only | `tests/__pycache__/test_gemini_api.*` ? no live source exists |
330
```

10. user (2026-06-10T06:08:22.546Z)

Structured output provided successfully

11. assistant (2026-06-10T06:08:28.473Z)

Verification complete. The finding's facts hold but it restates a documented, tracked deferral (docs/CODE_MAP.md:251 and :328, docs/BACKLOG.md:79), so it was rejected as not real at the claimed severity.